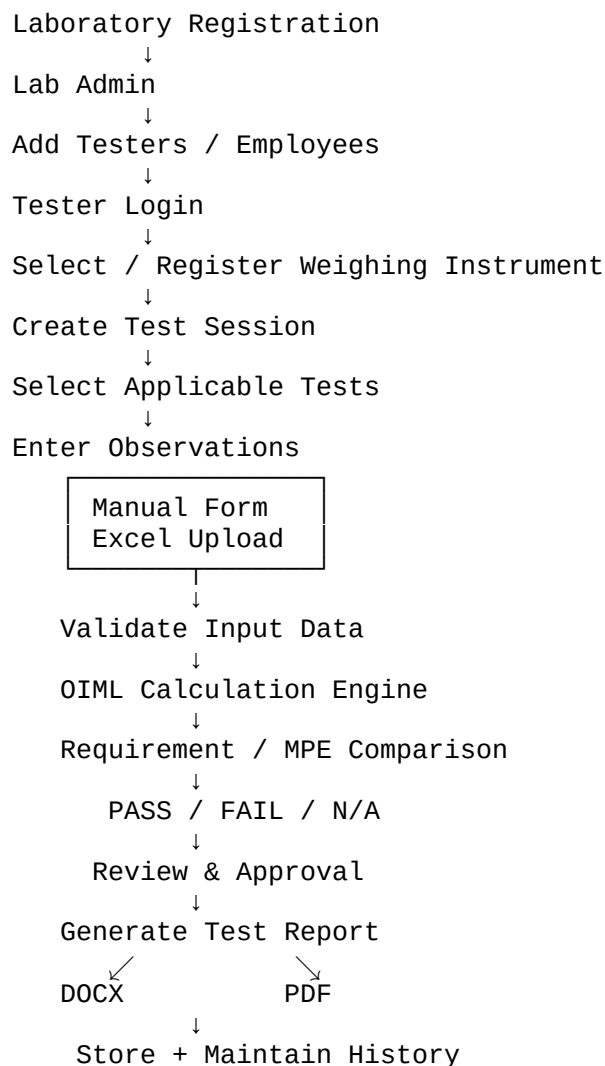


NAWI Test Report Generation System — Project Architecture & Specification

1. Project Overview

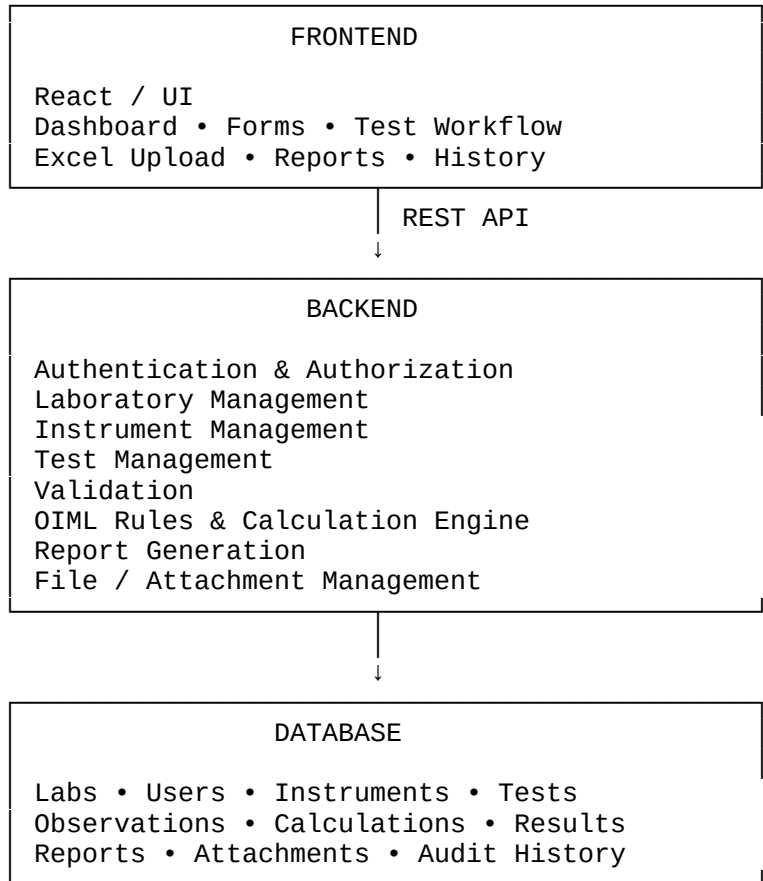
The project is a **web-based software system for testing Non-Automatic Weighing Instruments (NAWI)** according to **OIML R76** and generating standardized test reports.

Core workflow



2. Overall Website Architecture

Use a **3-tier web architecture**.



3. Main Website Modules

A. Authentication & User Management

Handles:

- Laboratory registration
- Lab administrator account
- Tester/employee creation
- Login/logout
- Role-based access
- Password management
- User activation/deactivation

Roles

Lab Admin

- Manage laboratory
- Add/remove testers
- View tests/reports
- Manage users

Tester

- Register/select instruments
- Create tests
- Enter observations
- Upload Excel
- Generate/submit reports

Reviewer / Approver

- Review results
 - Check test reports
 - Approve/reject
-

4. Laboratory Management

A laboratory is the main organizational unit.

Flow

```
Register Laboratory
  ↓
Create Lab Admin
  ↓
Lab Admin Dashboard
  ↓
Add Testers
  ↓
System generates credentials
  ↓
Tester Login
```

All users, instruments, tests and reports are associated with their laboratory.

This provides **laboratory-level data isolation**.

5. Instrument Management

The tester can:

Option 1 — Select Existing Instrument

Start New Test
↓
Select Existing Instrument
↓
Instrument specifications automatically loaded

Option 2 — Add New Instrument

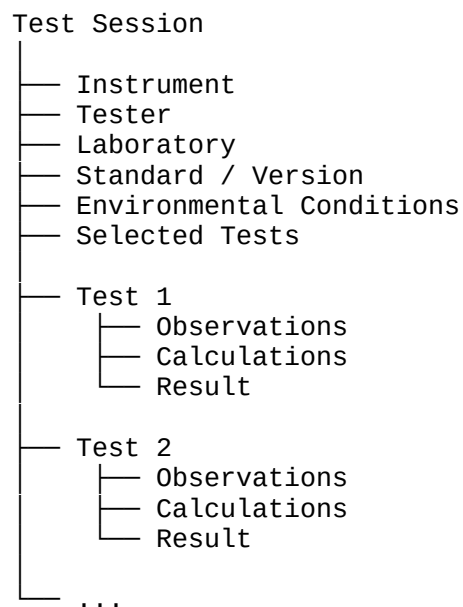
Add Instrument
↓
Enter specifications
↓
Save Instrument
↓
Available for future tests

Instrument information should therefore be maintained separately from individual test sessions.

This avoids repeatedly entering the same specifications.

6. Test Session Architecture

A **Test Session** represents one testing event for one weighing instrument.



A session should have states such as:

DRAFT
↓
IN PROGRESS
↓
SUBMITTED
↓
UNDER REVIEW
↓
APPROVED / REJECTED

7. Test Selection & Applicability

When a tester starts a test, the system determines which tests apply to the instrument.

Instrument Specifications
↓
Applicability Rules
↓

Test A → Applicable
Test B → Applicable
Test C → N/A
Test D → Applicable

N/A tests must not simply disappear.

They should remain visible in the test session and final report as:

Test: XYZ
Status: N/A
Reason: Not applicable to this instrument configuration

8. Observation Input System

The tester gets two methods of entering test data.

Method A — Manual Form

Select Test
↓
System displays required fields
↓
Tester enters observations
↓
Validation
↓

Save

The form should be **test-specific** rather than one giant generic form.

Method B — Excel/File Upload

```
Upload Excel
↓
Read file
↓
Validate columns + values
↓
Show preview/errors
↓
Tester confirms
↓
Save observations
```

Important: uploaded Excel data should pass through validation before entering the database.

9. OIML Rules & Calculation Engine

This is the core technical engine.

```
Observations
+
Instrument Specifications
+
OIML Rules
↓
Calculation Engine
↓
Calculated Result
↓
Requirement / MPE
↓
PASS / FAIL
```

The calculation engine should be **deterministic/rule-based**.

For example:

```
Load = 10 kg
Indication = 10.01 kg

Error = Indication - Load
```

$$= +0.01 \text{ kg}$$

Applicable MPE = $\pm 0.01 \text{ kg}$

$$|\text{Error}| \leq \text{MPE}$$

↓

PASS

The formulas, limits, clauses and applicability rules should be stored as **versioned configuration/rule data**, not hard-coded throughout the frontend.

10. Validation Layer

There should be validation at multiple levels.

Input Validation

Checks:

- Required fields
- Numeric values
- Units
- Valid ranges
- Missing observations
- Invalid Excel columns
- Duplicate/invalid data

Test Validation

Checks:

- Required observations present
- Correct test configuration
- Applicable rules available
- Required equipment/data available

Result Validation

Checks:

- Calculation completed
- Applicable requirement exists
- PASS/FAIL determination is valid

11. Report Generation

The report is the **main output of the system**.

Completed Test Session
↓
Validate Session
↓
Calculate Final Results
↓
Generate Report Data
↓
Report Template
↓

DOCX Report PDF Report

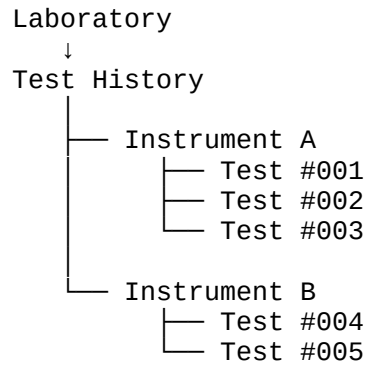
The report should contain, as applicable:

- Laboratory information
- Instrument information
- Tester information
- Test equipment
- Environmental conditions
- Applicable standard/version
- Individual test results
- Calculated values
- Requirements/MPE
- PASS/FAIL/N/A
- N/A reasons
- Remarks
- Attachments/references
- Approval/signature information

The **DOCX and PDF** should be generated from the same structured report data, so they remain consistent.

12. Report & History Management

Every completed test should remain accessible.



Users should be able to:

- Search reports
- Filter by instrument
- Filter by date
- Filter by tester
- Filter by result
- Open previous test sessions
- Download DOCX
- Download PDF
- View previous versions

Reports should be **versioned**, rather than overwritten.

13. Dashboard

The dashboard should provide a high-level view of laboratory activity.

Example:

Total Tests	124
Completed	98
Pending Review	12
Failed Tests	8
Draft Tests	6

Recent Tests

Instrument | Tester | Date | Status

The dashboard should primarily **derive its information from existing test/report data**, rather than maintaining a separate duplicate dataset.

14. Attachments

Tests should support attachments such as:

- Instrument photographs
- Nameplate photographs
- Test setup photographs
- Calibration certificates
- Excel input files
- Supporting documents

Attachments should be linked to the relevant test/session.

15. Audit & Security

Important actions should be recorded.

User
↓
Action
↓
Audit Log

Examples:

LOGIN
CREATE_TEST
UPDATE_INSTRUMENT
UPLOAD_EXCEL
SUBMIT_TEST
APPROVE_REPORT
GENERATE_REPORT
DOWNLOAD_REPORT

This gives the system a traceable history of changes.

16. Recommended Technical Architecture

A practical stack:

Frontend
React

- React Router
- Forms / Validation
- Dashboard
- API Client

↓ REST API

Backend
Python + FastAPI

- Authentication
- Laboratory APIs
- User APIs
- Instrument APIs
- Test APIs
- Calculation Engine
- Excel Processor
- Report Generator

↓

Database
PostgreSQL

↓

File Storage

- DOCX
- PDF
- Excel
- Attachments

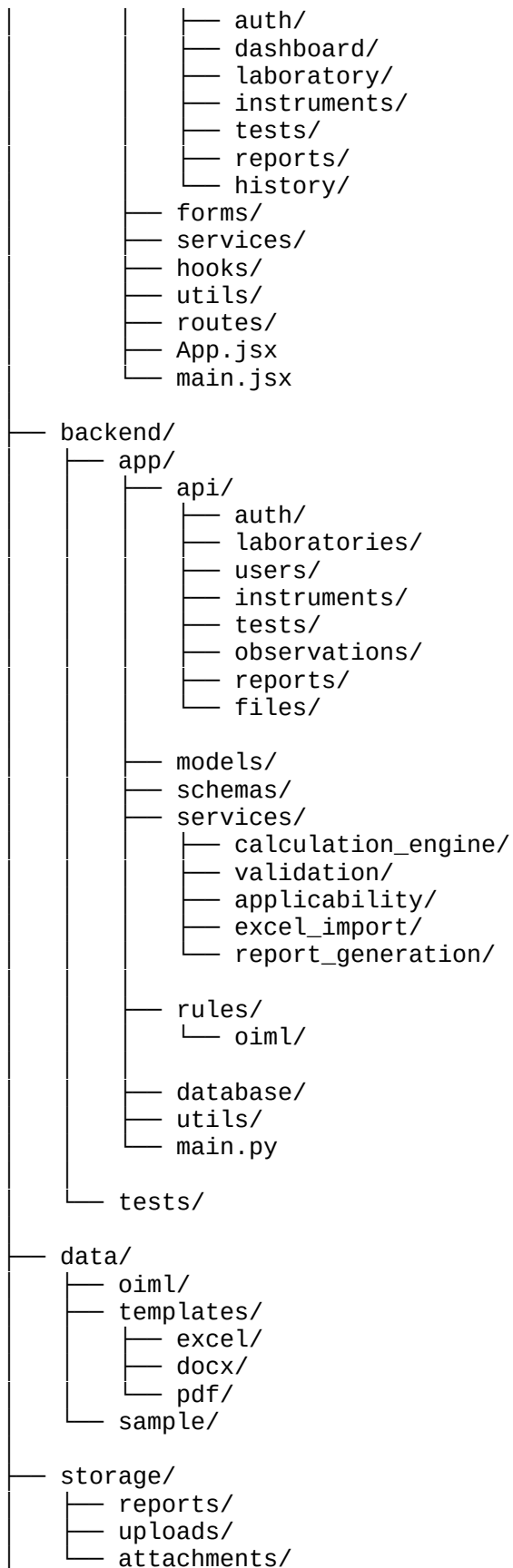
For the prototype, local file storage can be used; production can move this to object storage.

17. Project Directory Structure

Use one common structure for the entire team:

NAWI-Test-Report-System/

- frontend/
 - public/
 - src/
 - assets/
 - components/
 - layouts/
 - pages/



```
|
|— docs/
|   |— architecture/
|   |— api/
|   |— testing/
|   |— project/
|— scripts/
|— .env.example
|— .gitignore
|— README.md
|— docker-compose.yml
```

18. Important Development Rule

Everyone should follow this separation:

```
Frontend
  ↓
API
  ↓
Backend Services
  ↓
Database / Rules / File Storage
```

Frontend should not contain OIML calculation logic.

Database should store data, not application logic.

OIML rules/calculations should stay inside the backend rules/calculation layer.

Reports should be generated from stored structured test data, not directly from frontend form data.

This gives the whole team one consistent architecture while allowing everyone to work on different modules without creating conflicting implementations.